

REQUERIMIENTO 2

Prompt 1

realiza una auditoria al proyecto y dime en qué estado nos encontramos de las implementaciones incrementos-2-a-5-speckit.md, dime en que parte estamos y que implementaciones nos faltan

Respuesta Prompt 1

Se ha realizado una auditoria exhaustiva del proyecto contrastando el código, las migraciones, las pruebas y los artefactos de especificación contra `incrementos-2-a-5-speckit.md`.

1. Resumen Ejecutivo del Estado Actual

- **En qué parte estamos:**
 - El **Incremento 2** se encuentra 100% implementado, probado y completado en la rama activa `implementacion-2`.
 - Las **54 pruebas automatizadas** del suite (`pytest -v`) están pasando al 100% (29 de pruebas del Incremento 1 + 18 pruebas nuevas de servicios y 7 de integración).
- **Fase en la que nos encontramos:**
 - Listos para iniciar el Incremento 3 (Organización y priorización).

2. Estado Detallado por Incremento

- **Incremento 1:** Completado (HU-01 a HU-04, HU-12, HU-13). Registro, Login/Logout, creación de tareas, listado aislado por usuario, edición y máquina de estados básica.
- **Incremento 2:** Completado (HU-05, HU-06, HU-14).
 - HU-05: Soft delete con campo `deleted_at`.
 - HU-06: Reapertura de tareas completadas.
 - HU-14: Recuperación de contraseña mediante tokens.
- **Incrementos 3, 4 y 5:** Pendientes (0%). No iniciados.

Prompt 2

examina el estado del proyecto, vamos a empezar a realizar más implementación en el proyecto. Revisa los archivos y dame un breve resumen del estado del proyecto, acabe de agregar un archivo llamado incrementos-2-a-5-speckit.md el cual contiene el contenido de las implementaciones que nos faltan en el proyecto. no realices cambios por ahora, solo realiza la auditoria al proyecto, también agregue un archivo de readme-1.txt

Respuesta Prompt 2

1. Resumen Ejecutivo del Estado del Proyecto

- **Incremento 1:** Completado (100%). Implementadas HU-01 a HU-04 y HU-12, HU-13 (Auth + CRUD básico de tareas).
- **Pruebas Automatizadas:** 29/29 Pasando. 100% de tests unitarios y de integración exitosos en `tests/`.
- **Base de Datos y Migraciones:** Al día (Head). Esquema inicial generado con Alembic/Flask-Migrate.
- **Git & Working Tree:** Limpio. Rama `implementacion-1`, sin modificaciones pendientes en git.
- **Arquitectura:** Conforme. Respeta estrictamente los principios de la constitución (Monolito en capas: modelos, servicios, rutas y plantillas).

2. Estructura y Capas de la Aplicación

La arquitectura actual cumple con la separación estricta de responsabilidades:

1. **Modelos** (`src/models/`): `User`, `Task`, `AuditLog`.
2. **Servicios de Dominio** (`src/services/`): `TaskService`, `AuthService`, `AuditService`.
3. **Controladores / Rutas** (`src/routes/`): `auth_routes.py`, `task_routes.py`, `decorators.py`.
4. **Presentación** (`src/templates/` y `src/static/`): Plantillas Jinja2 y estilos CSS/JS estructurados.

Prompt 3

Vamos a realizar la implementación 2 del proyecto, analiza las tareas de la implementación 2 "Estructura y Capas de la aplicación" verifica las tareas y realiza la implementación con el plan completo

Importante: un compañero está realizando la implementación número 3, respeta las tareas que tenga la implementación 3 para no tener confusiones en la implementación de la rama número 3, revisa la implementación dos e incorporémosla.

Respuesta Prompt 3

Implementación 2 — Reporte de Finalización

- **Resultados de Pruebas:** 54 passed in 18.16s (29 Incremento 1 + 25 Incremento 2)
- **Lo que se implementó:**
 - **HU-05:** Soft Delete: campo `deleted_at` en `Task`, `delete_task()` en servicio, ruta y botón con confirmación JS.
 - **HU-06:** Reapertura: `reopen_task()` con auditoría diferenciada, ruta y botón solo en tareas completadas.
 - **HU-14:** Reset contraseña: modelo `PasswordResetToken`, rutas de request y reset, simulación por log.
- **Archivos clave creados/modificados:**
 - Nuevos: `password_reset_token.py`, `forgot_password.html`, `reset_password.html`, migración.
 - Modificados: `task.py`, `audit_log.py`, `task_service.py`, `auth_service.py`, `task_routes.py`, `auth_routes.py`, `list.html`.
- **Garantías para el Incremento 3:**
 - La rama `implementacion-2` es independiente de `implementacion-3`.
 - Los cambios son 100% aditivos: ningún archivo del Incremento 1 fue roto.
 - El campo `deleted_at` y la tabla no interfieren con `priority`, `categories` ni `due_date`.

REQUERIMIENTO 3

Prompt 1

revisa la estructura y todos los archivos .md de el proyecto mira que los requisitos esten bien y se esten cumpliendo para hacer el incremento numero 3 revisa el archivo incrementos-2-a-5-speckit.md

Respuesta Prompt 1

Estado General: Listos para el Incremento 3

54 tests pasan (100%) — los Incrementos 1 y 2 están sólidos.

Incremento 3 cubre: HU-07, HU-08, HU-09.

- **Lo que YA existe en el código que es relevante:**
 - `src/models/category.py` — ya existe pero está VACÍO.
 - `src/models/task.py` — tiene todo del Inc 1+2, necesita campos `priority` y `category_id`.
 - `src/services/task_service.py` — funciona, necesita `update_task_priority`, filtros y cálculo `is_overdue`.
- **Lo que hay que crear desde cero:**
 - `src/services/category_service.py`
 - `src/routes/category_routes.py`
 - Migración Alembic con la tabla `categories` + columnas `priority` y `category_id`.
- **Puntos clave a decidir antes de implementar:**
 1. Prioridad: ¿string o entero para facilitar el ORDER BY?
 2. `is_overdue`: ¿cómo se expone al template?
 3. Auditoría de prioridad: ¿separado de `TASK_UPDATED`?

Prompt 2

crea y haz la implementacion de el incremento numero 3 en base a el archivo incrementos-2-a-5-spec.md en este archivo estan todas las especificaciones y requerimientos para aprobar el incremento

Respuesta Prompt 2

Voy a leer primero las instrucciones de los skills de speckit antes de implementar.

El subagente de implementación está en ejecución. Mientras trabaja te explico qué está haciendo:

Plan de implementación del Incremento 3

- **Phase 1 — Modelos y Migración:** Implementar `Category` model, Extender `Task` con `priority` y `category_id`, Añadir constante `TASK_PRIORITY_CHANGED`, Añadir `@property is_overdue`, Generar y aplicar migración Alembic.
- **Phase 2+3 — Tests + Impl Prioridad:** Tests unitarios, `TaskService.update_task_priority()`, Extensión de `get_user_tasks()`, Ruta POST de prioridad.
- **Phase 4+5 — Tests + Impl Categorías:** Tests unitarios + integración de categorías, `CategoryService` completo, `TaskService.assign_category()`, Blueprint `category_bp`, Templates.
- **Phase 6+7 — Tests + is_overdue:** Tests cubriendo casos edge, Propiedad `@property is_overdue` integrada.
- **Phase 8 — Validación Final:** Suite completa `pytest -v`.

¡Excelente! El subagente terminó. Voy a verificar los resultados ejecutando los tests y revisando los archivos clave.

(5 files changed +696 -0)

Prompt 3

realiza el pytest de el incremento numero 3

Respuesta Prompt 3

Se ha ejecutado la suite de pruebas con `pytest` para el Incremento 3 (HU-07: Prioridad, HU-08: Categorías, HU-09: Tareas Vencidas).

Resultados de la ejecución:

Todos los 24 tests específicos del Incremento 3 han pasado exitosamente (100% de éxito).

REQUERIMIENTO 4

Prompt 1

Vamos a realizar la implementación 4 del proyecto, analiza las tareas de la implementación 4 "Colaboracion" verifica las tareas y realiza la implementación con el plan completo, asume que los incrementos del 1 al 3 ya estan implementados. no olvides guiarte de el documento de incrementos-2-a-5-speckit.md, de igual manera examina el proyecto para realizar la implementacion.

Importante: un compañero está realizando la implementación número 3, respeta las tareas que tenga la implementación 3 para no tener confusiones en la implementación de la rama número 3, revisa la implementación dos e incorpóremosla.

Respuesta Prompt 1

Implementé el Incremento 4 (HU-10 asignación y HU-11 notificaciones) en la rama local `implementacion-4`, creada desde `main`. Las 77 pruebas pasan (54 de los incrementos 1 y 2 y 23 nuevas), y la migración corre sin errores sobre una base vacía. No hice push.

- **Incremento 2:** Ya estaba integrado en `main`.
- **Incremento 3:** No existe en el repositorio y no lo toqué. Los cambios a archivos compartidos son pequeños y aditivos.
- **Decisiones de diseño:**
 - Propietario y asignado: `Task.user_id` sigue siendo el propietario y se agrega `Task.assignee_id`.
 - Asignación por correo: el correo debe estar registrado y se valida en el backend.
 - Listado: muestra las tareas creadas por el usuario y las asignadas a él.
 - Permisos del asignado: solo ve la tarea.
 - Notificaciones: se marcan como leídas y no se descartan.
 - Atomicidad: asignación, notificación y auditoría se guardan en un solo commit.
- **Endpoints nuevos:** POST para asignar, GET para notificaciones, POST para leer notificaciones.
- (18 files changed +703 -10)

Prompt 2

pueden implementar la rama implementacion-4 en el repositorio principal para dejarla en espera para fusionar primero la rama 3 y después la 4.

Respuesta Prompt 2

Subí la rama `implementacion-4` al repositorio remoto. No la fusioné con `main`, así que sigue en espera.

El orden de fusión queda así:

1. **Rama 3** → `main`.
2. **Rama 4** → `main`, después de actualizarla con `main`. Habrá que correr `flask db merge heads` o cambiar su `down_revision` para que apunte a la migración de la rama 3.
3. Revisar a mano `get_user_tasks` en `task_service.py` y `list.html`, donde ambas ramas pueden chocar. Después hay que correr `pytest` completo.

Prompt 3 (Corrección de desviación)

Caso donde se desvía (se hace la corrección por que aparecen 3 pruebas en peligro se corrigió pidiéndole a el agente que lo revisara y que solucionara lo que le pareciera conveniente)

ya realice las pruebas de la implementación numero 4 y las pruebas pasaron de manera exitosa, sin embargo tengo 3 pruebas en peligro, necesito que verifiques el estado de las pruebas que te voy a proporcionar en el archivo respuesta_pruebas.txt y soluciones los inconvenientes.

Respuesta Prompt 3

Las 3 pruebas "en peligro" eran advertencias de deprecación de SQLAlchemy (`LegacyAPIWarning`), no fallos. Ya las corregí y la suite pasa con 101 pruebas y 0 advertencias.

`Query.get()` está obsoleto desde SQLAlchemy 2.0 y aparecía en tres líneas de las pruebas de la rama 3. Lo cambié por `db.session.get(Modelo, id)` en:

- `test_category_routes.py`, líneas 47 y 48.
- `test_task_routes.py`, línea 133, donde también importé `db`.

El cambio está en un commit de la rama local `fix/query-get-warnings`, creada para no tocar `main`. No hice push.

REQUERIMIENTO 5

Prompt 1

Vamos a realizar la implementación 5 del proyecto, analiza las tareas de la implementación 5 "Interfaz e interacción (JavaScript)" verifica las tareas y realiza la implementación con el plan completo, asume que los incrementos del 1 al 4 ya están implementados. no olvides guiarte de el documento de incrementos-2-a-5-speckit.md, de igual manera examina el proyecto para realizar la implementación.

Importante: Realizar la implementación número 5 respetando la jerarquía de las implementaciones anteriores.

Respuesta Prompt 1

Implementación 5 lista

Analicé HU-15 y HU-16 del documento y añadí la especificación, el plan técnico y las tareas del incremento, todas marcadas como completadas.

- **Completar sin recargar:** la interfaz usa `fetch` con el endpoint de estado existente. Actualiza el badge de forma optimista, pero revierte el cambio y anuncia el error si no recibe confirmación del backend.
- **Reordenamiento persistente:** añadí `Task.position`, el servicio y POST `/tasks/reorder`. El arrastre se habilita solo para tareas propias en una vista sin filtros; el backend valida propiedad y exige el conjunto completo de tareas activas. La migración conserva el orden actual de las tareas existentes.

Validación: `pytest -q` pasó con 110 pruebas; `node --check` y `git diff --check` finalizaron sin errores. También ejecuté la migración en una base temporal y la eliminé al terminar.

(13 files changed +680 -25)

AUDITORÍA COMPLETA

Prompt 1

Necesito que realices una auditoria completa de el proyecto y de todos los errores que pueda tener de este tipo, necesito que revises la jerarquia de los archivos y que no coquen las implementaciones entre si, en la auditoria que vas a realizas crea un archivo de pruebas y revisa que todo este funcionando perfectamente sin romperse, deja todo el funcionamiento de la aplicacion guiandoote de los archivos de planes e incrementos, utiliza las skills de la carpeta .agents de ser necesario.

Respuesta Prompt 1

La auditoría está hecha. La app ya funciona de extremo a extremo: 183 pruebas pasan (110 existentes más 73 nuevas). Comprobé además con un servidor real en `127.0.0.1` que registro, login, creación de tareas y las páginas principales responden bien. Todo está en la rama `auditoria-completa`, creada desde `implementacion-5`.

Causa del error de tu captura:

`taskcontrol.db` estaba vacío (0 bytes), así que la tabla `users` no existía. Además, el esquema no se podía construir ni con `flask db upgrade`, porque faltaba la migración del Incremento 3. Añadí la migración encadenada al final de la historia.

Otros errores corregidos:

- **Código duplicado por los merges:** `Task` tenía definiciones dobles (`is_overdue`, etc.).
- **Redirección abierta en el login:** redirigía fuera de la app.
- **Códigos HTTP:** los JSON devolvían 400 cuando la tarea no existía; ahora devuelven 404.
- **Edición por un tercero:** editar con formulario una tarea ajena provocaba un error 500; ahora redirige al listado.
- **Fecha y Categoría inválidas:** devolvían errores no controlados, ahora responden 400.
- **Título y descripción:** validaciones añadidas para rechazar formatos inválidos (JSON) y títulos de más de 200 caracteres.

Pruebas de auditoría (`test_auditoria_completa.py`):

Cubre estructura, rutas, migraciones y flujos por incremento (del 1 al 5). Comprobé que 9 de esas pruebas fallan sobre el código original y pasan con las correcciones.

(4 files changed +673 -4)

CONCLUSIÓN Y NOTAS DE CIERRE

En el proyecto como tal tuvo muy pocas desviaciones por no decir ninguna. Se asumió que el proyecto se desarrolló bien ya que las pruebas fueron satisfactorias. Sin embargo, al ejecutarse todas las pruebas con el Incremento 5 finalizado, se ejecutó una auditoría de prueba por prueba en cada incremento otra vez, realizando las respectivas correcciones al finalizar. Al dar un resultado satisfactorio en las pruebas funcionales, se asume que **el proyecto fue un éxito.**